

IIT BOMBAY
Course Name: HDL

Project Report: Design and Simulation of a 2-Way Set-Associative Cache Controller

1 Introduction

1.1 What is a Cache and Why is it Used?

In modern computer architecture, there is a significant speed disparity between a fast Central Processing Unit (CPU) and relatively slow Main Memory (RAM). To prevent the CPU from constantly stalling while waiting for data, a smaller, much faster memory structure called a **Cache** is placed between them. The cache temporarily stores copies of the most frequently used main memory data. When the CPU needs data, it checks the cache first (a “hit”). If the data is absent (a “miss”), it is fetched from main memory and stored in the cache for future use.

1.2 Project Abstract

This project details the design, implementation, and simulation of a single-level Cache Controller using hardware description language (Verilog). The cache features a **2-Way Set-Associative** architecture, a Write-Around policy for write misses, a Write-Back policy for write hits, and utilizes a Least Recently Used (LRU) algorithm for block eviction.

1.3 Understanding 2-Way Set-Associative Cache

A 2-Way Set-Associative cache is a compromise between a Direct-Mapped cache (where each memory block maps to only one specific line, causing frequent collisions) and a Fully-Associative cache (where a block can go anywhere, requiring complex and slow search logic). In our design, the cache is divided into 32 **Sets**, and each set contains 2 **Ways** (slots). When a memory block is mapped to a specific set, the controller can place the data in either Way 0 or Way 1. This significantly reduces conflict misses while keeping the search hardware relatively simple.

2 Least Recently Used (LRU) Replacement Policy

Because each set in our cache has two available slots (Ways), the controller requires a set of rules to determine which slot to overwrite when a set becomes completely full. This project implements the **Least Recently Used (LRU)** policy.

The LRU policy assumes that data accessed recently will likely be accessed again soon, while data that has been ignored for a long time is safe to evict. In our 2-way architecture, this is achieved highly efficiently using a single bit per set:

- If the LRU bit is 0, it indicates that **Way 0** is the oldest and should be evicted next.
- If the LRU bit is 1, it indicates that **Way 1** is the oldest and should be evicted next.

Every time the processor successfully reads or writes to a way, the controller flips the LRU bit to protect that way from being overwritten in the next cycle.

3 Test Cases and Simulation Results

To verify the logical correctness of the controller, a robust testbench was developed to force collisions within a single set (Set 0). The following test cases demonstrate the progression of data loading, LRU tracking, eviction, and writing.

3.1 Test 1: Read Miss (Filling Way 0)

The processor requests data from Address 128 (Block 32). The controller checks Set 0 and finds it empty, resulting in a **Read Miss**. The controller stalls the processor for 10 clock cycles, fetches Block 32 from main memory, and places it into **Set 0, Way 0**. The LRU bit is updated to protect Way 0.

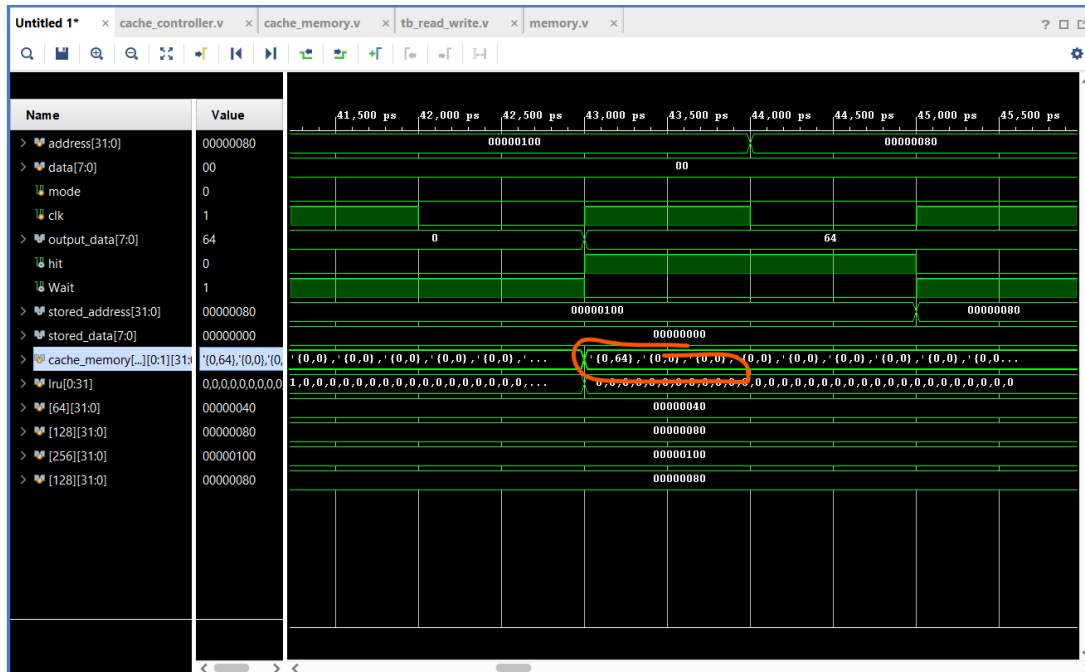


Figure 1: Waveform demonstrating Test 1: Block 32 loading into Way 0.

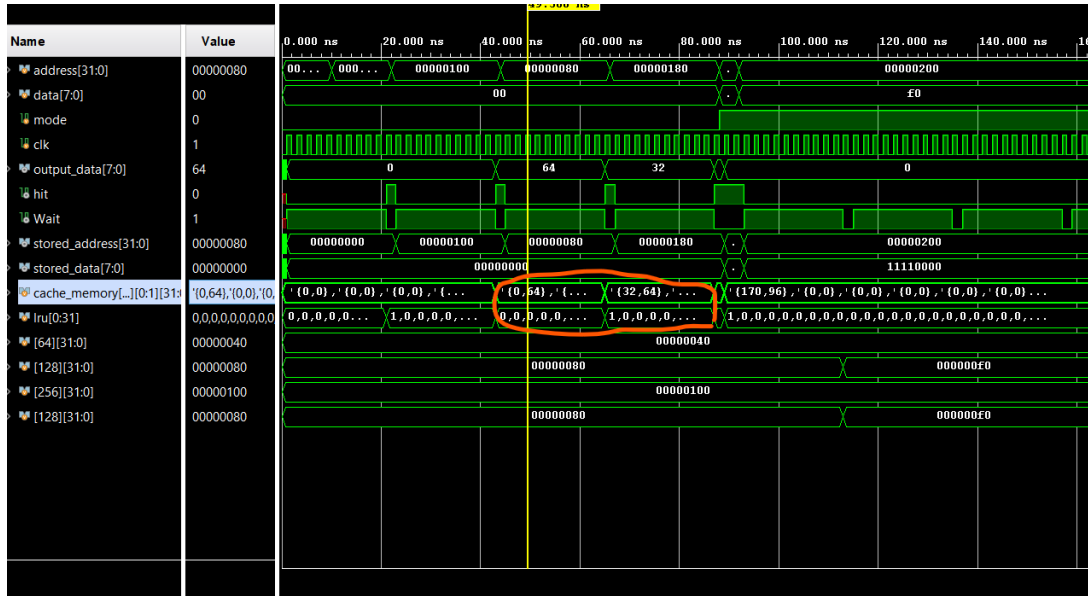


Figure 3: Waveform demonstrating Test 3: Read Hit on Way 0.

3.4 Test 4: Read Miss with Eviction

The processor requests Address 384 (Block 96). Set 0 is full. This is a **Read Miss**. The controller checks the LRU bit and determines that Way 1 (Block 64) is the oldest. Block 64 is safely evicted back to main memory, and Block 96 is loaded into **Way 1**.

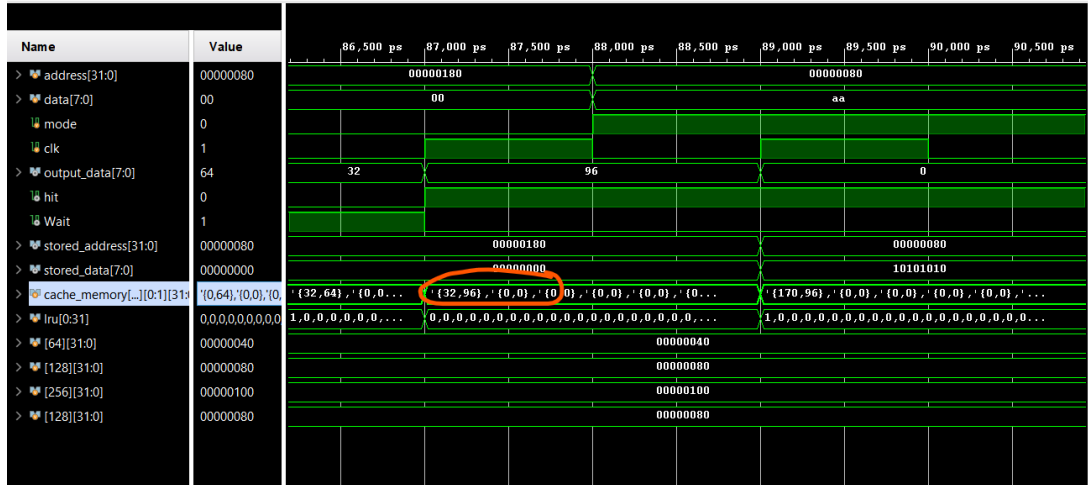


Figure 4: Waveform demonstrating Test 4: Block 64 evicted, Block 96 loaded into Way 1.

3.5 Test 5: Write Hit

The processor attempts to write the hexadecimal data AA to Address 128 (Block 32). The controller finds Block 32 sitting in Way 0, resulting in a **Write Hit**. The new data is written instantly into the cache array without stalling the processor.

